

# Origin Object Oriented Techniques (OOT) (UNIT-1)

①

## Introduction to Object Oriented Programming

### Introduction:-

Traditional programming language such as COBOL, FORTRAN etc. are known as procedure oriented language. Program written in these languages had some disadvantages like the length of the code was difficult to manage. Another main problem is data security.

In general, conventional programming language had the following disadvantages:-

→ Data was not secure as data passes globally from one function to another function globally.

### Programming Paradigms:-

Generally, following programming paradigms are frequently used:-

#### 1. Monolithic Programming:-

↳ In this approach, data variables were global in nature.

↳ The statements were executed in sequence. However, the control can be transferred using JUMP statements.

↳ Main problem was that data was not secure due to its global scope.

↳ It was suitable for smaller application as concept of subroutines was not possible. Ex - BASIC, ASSEMBLY.

#### 2. Procedural Programming:-

↳ In this method, the program was divided into no. of segments.

↳ Here also, the data was not secure as once again the data was global.

↳ For control of program, again jump statements were used.

↳ It was useful for medium sized applications.

Ex - FORTRAN, COBOL.

①

### 3. Structured Programming :-

- ↳ It is useful for large sized applications.
- ↳ Here, the program is divided into no. of sub modules. Each module is responsible for a separate task.
- ↳ Data is a bit more secure as the concept of local scope is introduced.
- ↳ This approach introduced the concept of data types which made the program more convenient. Ex, PASCAL, C.

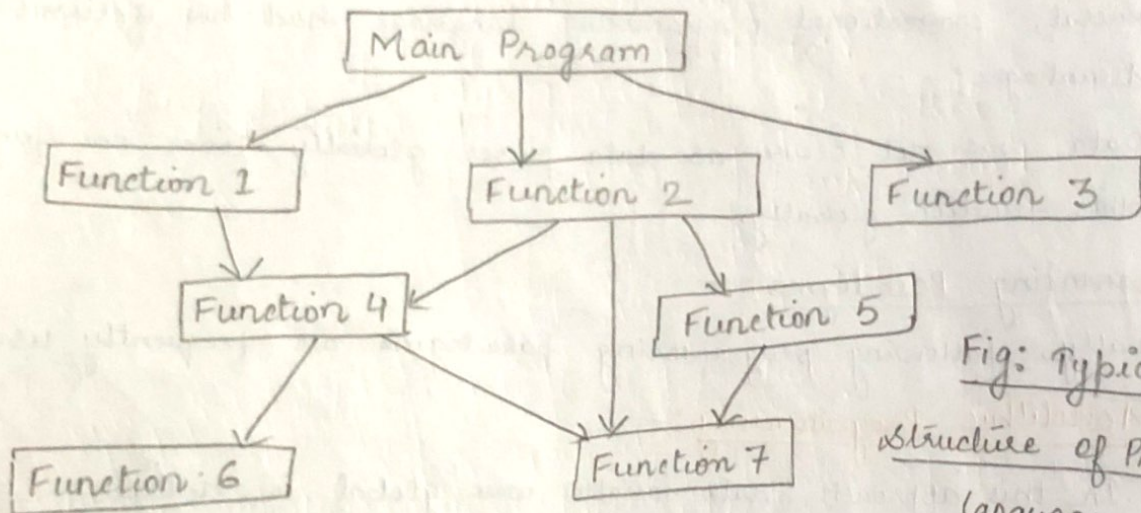


Fig: Typical  
Structure of Program  
Language

### 4. Object Oriented Programming :-

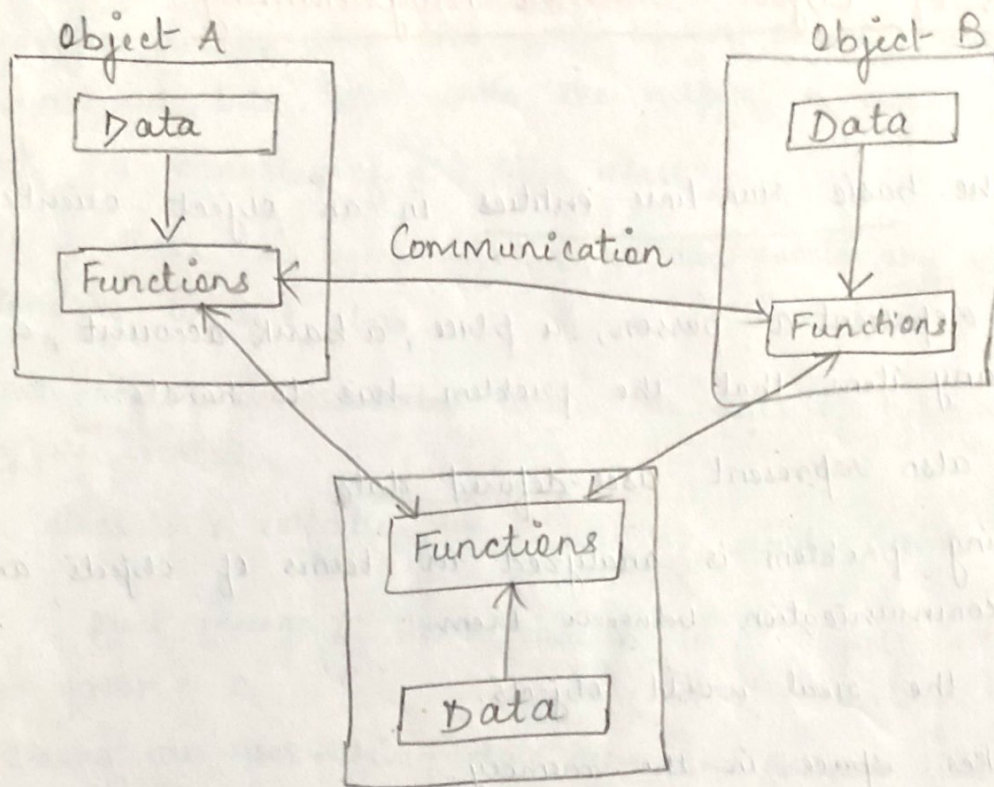
The major motivating factor in object oriented approach is to remove some of the problems encountered in procedural approach.

In object oriented programming :-

- It treats data as a critical element in the program development and does not allow it to flow freely around the system.
- It ties data more closely to functions that operate on it and protects it from accidental modification from outside functions.
- It allows decomposition of problem into no. of entities called objects and builds data and function around these objects.

NOTE- It is the most recent concept among programming paradigms and still means different things to different people.

## Organization of data and functions in Object oriented programming



### Definition

It is an approach that provide a way of modularizing programs by creating partitioned memory area for both data and functions that can be used as templates for creating copies of such modules on demand.

### Characteristics :-

- i) Emphasis is on data rather than procedure.
- ii) Programs are divided into objects.
- iii) Data structures are designed such that they characterize the objects.
- iv) Functions that operate on the data of an object are tied together in the data structure.
- v) Data is hidden and cannot be accessed by external functions.
- vi) Objects may communicate with each other through functions.
- vii) New data and functions can be easily added.
- viii) It follows bottom up approach in program design.

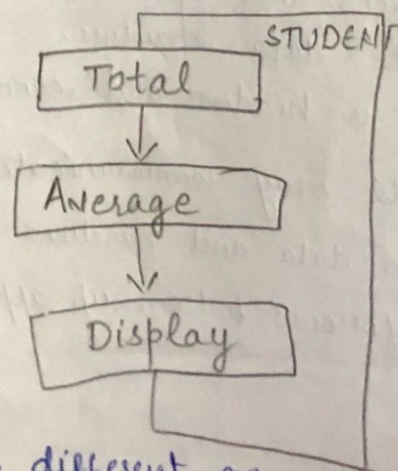
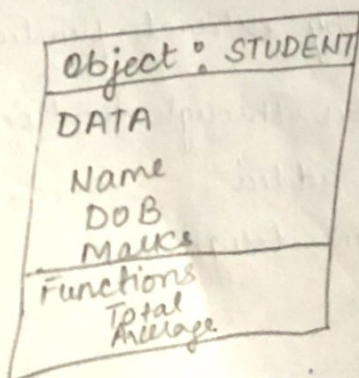
## Key concepts of Object-Oriented Programming :-

### 1. Objects :-

- i) objects are the basic run-time entities in an object-oriented system.
- ii) They may represent a person, a place, a bank account, a table of data or any item that the problem has to handle.
- iii) They may also represent user-defined data.
- iv) Programming problem is analyzed in terms of objects and the nature of communication between them.
- v) It matches the real world objects.
- vi) Objects takes space in the memory.
- vii) It can be uniquely recognized by its name.
- viii) It declares the state shown by data values of its characteristics at a specific run time. The state of object varies according to procedure used. It is known as action of object.

e.g. let "customer" and "account" are two objects in a program, then the customer object may send a message to the account object requesting for a bank balance. Each object contains data, code to manipulate data.

Objects can interact without having to know details of each other's data or code.

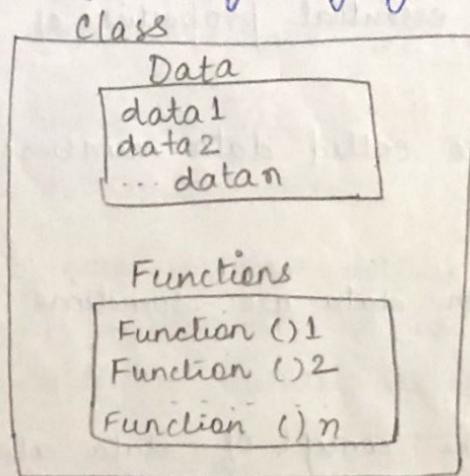


Representation of methods in different manners.

④

## Classes :-

- i) The entire set of data and code of an object can be made a user-defined data type with the help of a class.
- ii) Objects are variables of the type class.
- iii) Once a class has been defined, we can create any no. of objects belonging to that class.
- iv) Each object is associated with the data of type class with which they are created.
- v) A class is a collection of objects of similar type.  
e.g. fruit mango; Here mango is a object belonging to the class fruit.
- vi) classes are user-defined data types and behave like the built-in types of a programming language.



← e.g. of class.

E.g.

class : car

Properties : model, company, color, capacity

Actions : Speed(), average() and break().

class : computer

Properties : brand, price, hard disk, ram size

Actions : processing(), display() & printing()

### 3. Method:-

- i) An operation required for an object or entity when coded in a class is called a method.
- ii) Operation required for an object are to be defined in a class.
- iii) All objects in a class perform certain common actions or operation.
- iv) Each action needs an object that becomes a function of the class that defines it and is referred to as a method.

### 4. Data Abstraction:-

- i) Abstraction refers to the act of representing essential features without including the background details.
- ii) Classes use the concept of abstraction and are defined as a list of abstract attribute and functions to operate on these attributes.
- iii) They encapsulate all the essential properties of objects that are to be created.
- iv) The attributes are sometime called data members because they hold information.
- v) Functions that operate on data are sometime called methods or member functions.
- vi) Since the classes use the concept of data abstraction they are known as abstract data type (ADT).

### 5. Encapsulation:-

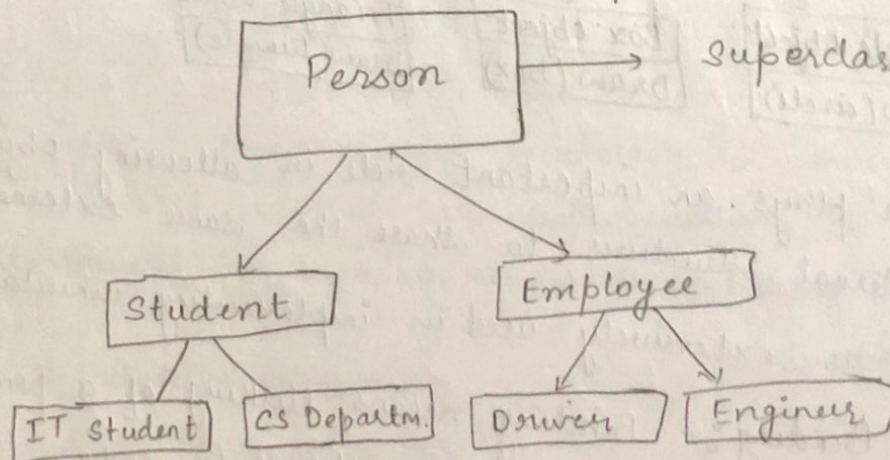
- i) The wrapping up of data and functions into a single unit is called encapsulation.
- ii) Data encapsulation is the most striking feature of class.
- iii) The data is not accessible to the outside world, and only those functions which are wrapped in the class access it.

(6)

- iv) These functions provide to interface b/w the object's data and the program.
- v) The insulation of data from direct access by the program is called data hiding or information hiding.

## 6. Inheritance:-

- i) It is the process by which objects of one class acquire the properties of objects of another class.
- ii) It supports the concept of hierarchical classification
- iii) The concept of inheritance provides the idea of reusability
- iv) we can add additional features to an existing class without modifying it.
- v) This is possible by deriving a new class from the existing one. The new class will have the combined features of both the classes.
- vi) The real power of inheritance mechanism is that it allows the programmer to reuse a class.
- vii) Note that each sub-class defines only those features that are unique to it. Without the use of classification, each class would have to explicitly include all of its features.



## 7. Polymorphism :-

It means the ability to take more than one form & operation may behave differently in different instances.

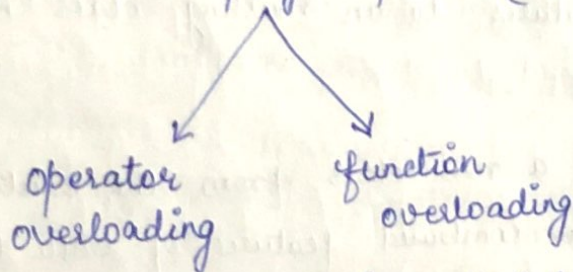
The behavior depends upon the types of data used in operation.

For e.g. operation of addition behave differently if we give the two operands int or string type.

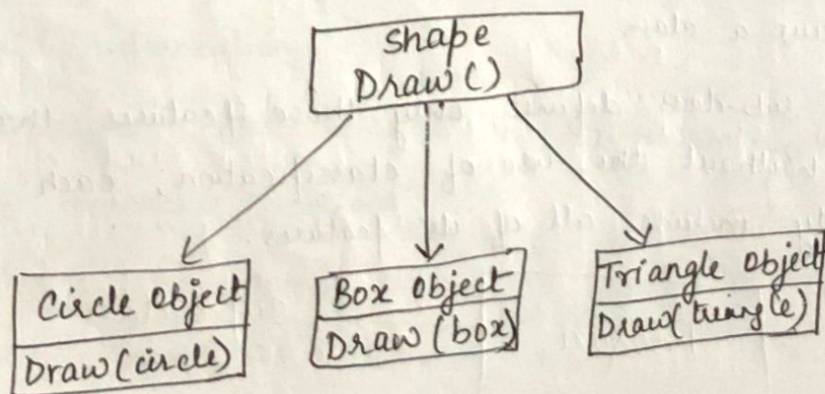
In the int type case, addition operation add the two integer number and in string it concatenates the string.

### Types of Polymorphism :-

#### i) Compile time polymorphism (early binding)



#### ii) Run time polymorphism (late binding)



Polymorphism plays an important role in allowing objects having different internal structures to share the same external interface.

Polymorphism is extensively used in implementing inheritance.

## 6. Dynamic Binding :- Binding means linking of a procedure code to the code to be executed in response to code

Dynamic binding means the code associated with the given procedure code at the Runtime.

### 9. Message Passing :-

It means make the communication b/w the objects or communication establish b/w object by receiving or sending information.

Message passing involves :-

- i) creating class that define objects & their behavior.
- ii) Establishing communication among objects.
- iii) creating objects from class definition

A message for an object is a request for execution of a procedure. ∴ It will invoke (call) a function in the receiving objects that generates the desired result. It means message passing involves specifying the name of the object & name of the function & name of the info to send.

e.g. →

Employee.salary(name);

Object                      message                      Info.

Objects have a life cycle. They can be created and destroyed. Communication with an object is feasible as long as it is alive.

### 10. Generosity :-

It is a property of Object-oriented technology in which the container class can be created in which the implementation of any data type is contained. The container classes are popularly known as templates.

Ex<sub>2</sub>

```
#include <iostream.h>

template <class T>
void show(T x)
{
    cout << " x = " << x;
}
```

```
void main()
{
    int i = 10;
    float f = 12.5;
    char c = 'A';
    show(i);
    show(f);
    show(c);
}
```

9

Above program will show output as: -

$x = 10$

$x = 12.5$

$x = A$

## Benefits of OOP :-

- ↳ Software complexity can be easily managed.
- ↳ Object oriented can be easily upgraded from small to larger systems.
- ↳ Through inheritance we can eliminate redundant code and extend the use of existing classes.
- ↳ We can build programs from standard working modules that communicate with one another.
- ↳ Data hiding helps programmer to build secure programs.

## MODELING :-

- Modeling is a well accepted engg. technique for designing the s/w. The architectural and mathematical models can be build for creating the software.
- Modeling is an activity, the aim of which is to make a particular part or feature of the system easier to understand, define, quantify, visualize or simulate.
- It is a specification of reality.
- It provides the blueprints of a system.

## Advantages of Modeling

- a) It helps us to visualize a system as it is or we want it to be.
- b) It permit us to specify the structure or behavior of a system.
- c) It gives us a template that guides us in constructing a system.
- d) It documents the decisions we have made.

(10)

## Importance of Modeling

Same as advantages.

## Principles of Modeling

1. The choice of what models to create has a profound influence on how a problem is attacked and how a solution is shaped.
2. Every model may be expressed at different levels of precision.
3. The best models are connected to reality.
4. No single model is sufficient: Every non trivial system is best approached through a small set of nearly independent models.

## MODEL:-

A model provides the blueprint of the system. A good model include these elements that have broad effect and omit those minor elements that are not relevant to a given level of abstraction.

### Aims of Modeling

1. We model the system because we want to understand the system which we are building.
2. Models help to visualize the system.
3. Models allow us to specify the structure or behavior of the system.
4. Models help in documenting the decisions that are made during the development of the software.
5. Using model we get the template, using which the system can be constructed.

## Object - Oriented Modeling

Generally while building software systems there are two commonly used approaches of modeling

1. Algorithmic Perspective
2. Object - Oriented Perspective.

The building block of algorithmic perspective is function or procedure. In this approach, the larger algo. is decomposed into smaller ones. As the requirements get changed as the system grows, it becomes very difficult to maintain the algo.

The modern software development approach is based on Object oriented Perspective. The main building block of this approach is object or class.

### Importance of OOP over Conventional approach:-

1. The conventional programming executes series of procedures sequentially. In object oriented programming approach there is a collection of objects.
2. Conventional programming is top-down whereas OOP is bottom-up programming approach.
3. In conventional programming, major focus is on procedures or functions. In OOP major focus is on objects.
4. In conventional programming, data reusability is not possible. In OOP data reusability is possible.
5. In conventional programming, data hiding is not possible. In OOP data hiding is possible.
6. Conventional programming is simple to implement. OOP is complex to implement.

## 1. Translating classes into structures

```
class Rectangle  
{  
  
}
```



```
struct Rectangle  
{  
  
}
```

## 2. Passing values / argument to methods

- call by value
- call by reference

## 3. Implementing Inheritance

## 4. Implementing Associations

## 5. Implementing storage for objects.

## Object Oriented Programming style :-

### Reusability

Software reusability comes from idea that a programming problem should only be solved once and from then on, the code for the solution simply copied into a project that needs it.

### Extensibility

In software engg, extensibility is a system design principle where the implementation takes into consideration for future growth.

It is a systematic measure of the ability to extend a system & level of effort required to implement the extensions.

Extensions can be through addition of new functionality or through modification of existing functionality.

### 3) Robustness

consider two of the main reasons for program failure: memory management mistakes & mishandled exceptional conditions.

#### memory management

deallocation is completely automatic, because Java C++ provides garbage collection for unused objects.

#### mishandled exceptional

all runtime errors can and should be managed by the program.

# UML (Unified Modelling Language)

## UNIT - I (OOT)

(2)

### Overview of UML

- General purpose Modelling language
- Main aim is to define standard way to visualize the way a system has been designed.
- Standard language for writing software blueprint.
- Provides set of notations to create model of system.
- Models are very useful in documenting design & analysis results.
- UML
  - Not a system design & development methodology
  - Not tied to any methodology.
- It can only be used to document object-oriented analysis and design results obtained using some methodology.
- It is one of the objective of developer of UML to keep the notations of UML independent of any specific design methodology.
- It is a graphical language for visualizing, specifying, constructing and documenting the artifacts of s/w intensive system.
- UML is appropriate for modeling systems ranging from enterprise info system to distributed web-based applications.

UML is a language for

#### 1. Visualizing :-

- bunch of graphical symbols
- behind each symbol in UML, notation is well-defined semantics.
- one developer can write a model in UML, another developer in another tool can interpret that model unambiguously.

(1)

## 2. Specifying :-

- Help in building models that are precise, unambiguous & complete.
- Used to understand the complete behaviour of the system.
- Also used for understanding functional & non-functional requirements.

## 3. Constructing :-

- UML is not a visual programming language, but its model can be directly connected to a variety of programming languages.

## 4. Documenting :-

- UML helps in documentation of all stages of software development life cycle like requirement gathering & analysis, specification, design, coding, testing, maintenance.

## UML Diagrams

UML can be used to construct 9 different types of diagrams to capture different views of a system.

## Where the UML can be used?

- Banking & Financial services
- Enterprise info. system
- Telecommunication
- Transportation
- Retail
- Defense/Aerospace
- Scientific
- Distributed web based services
- Real time Embedded system - video games, microwave oven, music players.

## Architecture of UML :-

A system architecture is perhaps the most important artifact that can be used to manage the different viewpoints and so control the iterative & incremental development of a system throughout its lifecycle.

Architecture is the set of significant decisions about

- The organization of a software system.
- The selection of the structural elements and their interfaces by which the system is composed.
- Their behaviour, as specified in the collaborations among these elements.
- The composition of these structural and behavioral elements into progressively larger subsystems.
- The architectural style that guides this organization: the static and dynamic elements and their interfaces, their collaboration and their composition.

Software architecture not only concerned with structure & behavior but also with

- ↳ usage
- ↳ Functionality
- ↳ Performance
- ↳ Resilience
- ↳ Reuse
- ↳ comprehensibility
- ↳ Economic & technology constraints
- ↳ Trade-offs
- ↳ Aesthetic concerns

Vocabulary (Functional req.)  
functionality

Design view

Implementation view

System assembly  
configuration  
Management

behavior

Process view

Deployment view

System topology  
distribution  
delivery  
installation

Performance  
scalability, throughput

③

Use case view

### Use case view

- It encompasses the usecase that describes the behavior of the system as seen by the end users, analysts & testers.

### Design view

- It encompasses the classes, interface & collaborations that form the vocabulary of the problem & its solution.
- It primarily supports functional requirements.

### Process view

- It encompasses classes, interface threads & processes that forms the system concurrency & synchronization.
- ↳ It primarily addresses the performance, scalability & throughput of the system.

### Implementation view

- It encompasses the components & files that are used to assemble and release the physical system.
- It addresses the configuration management of system.

### Deployment view

- It encompasses the ~~threads~~ nodes that form the system's n/w topology on which system executes.
- It addresses the distribution, delivery & installation of the parts that make up the physical system.

### Conceptual Model of UML :-

To understand the UML, we need to form a conceptual model of the language, and this requires learning three major elements :-

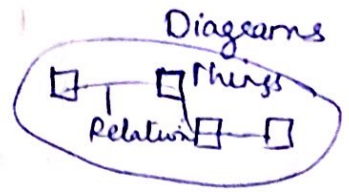
- ↳ UML's basic building blocks
- ↳ Rules that dictate how those building blocks may be put together
- ↳ Common mechanisms that apply throughout the UML.

(4)

## UML Basic Building Blocks

Three basic building blocks of UML:-

- ① Things - abstract entities
- ② Relationships - tied things together
- ③ Diagrams - combining things & relationships



↳ THINGS:- Basic and abstract in model. There are 4 types of things:-

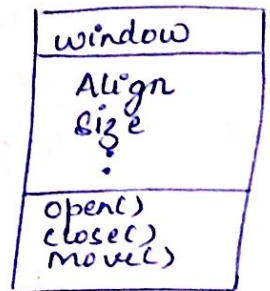
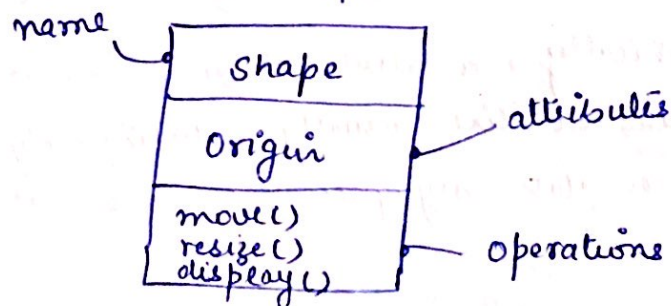
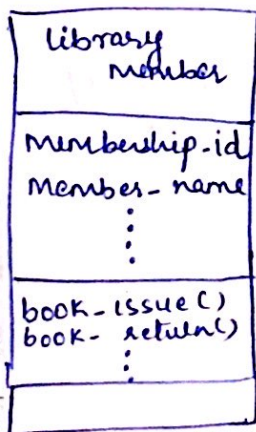
### ① Structural things:-

Static part of a model, representing ~~road~~ elements that are either conceptual or physical.

- classes
- interfaces
- collaborations
- use cases
- Active classes
- Components
- Nodes
- Objects

a) class:- set of objects having similar responsibilities

- a class is a description of a set of objects that share the same attributes, operations, relationships and semantics.
- class implements one or more interfaces.
- Class is rendered as a rectangle, usually including its name, attributes & operations.

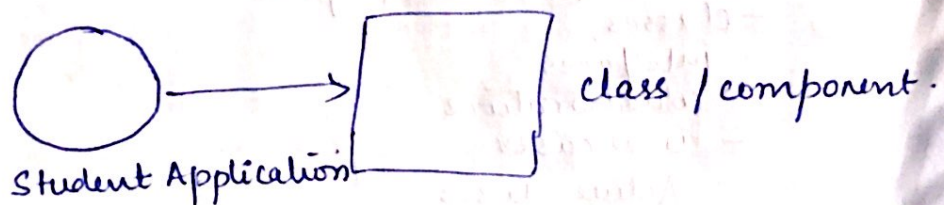


- Extra component

⑤

b) Interface define a set of operations which specify the responsibility of class.

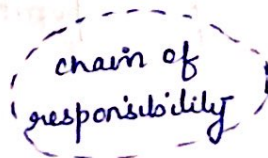
- It is used to describe the functionality without implementation
- Interface is a collection of operations that specify a service of a class or component.
- An interface therefore describes the externally visible behaviour of that element.
- An interface might represent the complete behavior of class or only a part of that behavior.
- An interface is rendered as a circle together with it's name.



- Attached to class or component that realizes the interface.

c) collaboration defines interaction between elements

- defines an interaction, which have structural as well as behavioural dimensions
- It is a society of roles and other elements that work together to provide some cooperative behaviour that's bigger than the sum of all the elements.
- Graphically, a collaboration is rendered as an ellipse with dashed line usually including only it's name.
- A given class may participate in several collaborations.

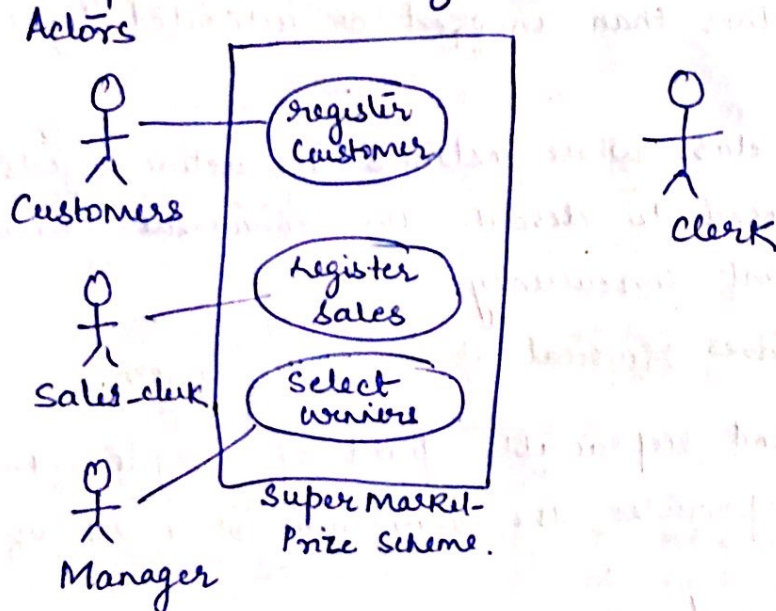


d) Use Case represents a set of actions performed by a system for a specific goal

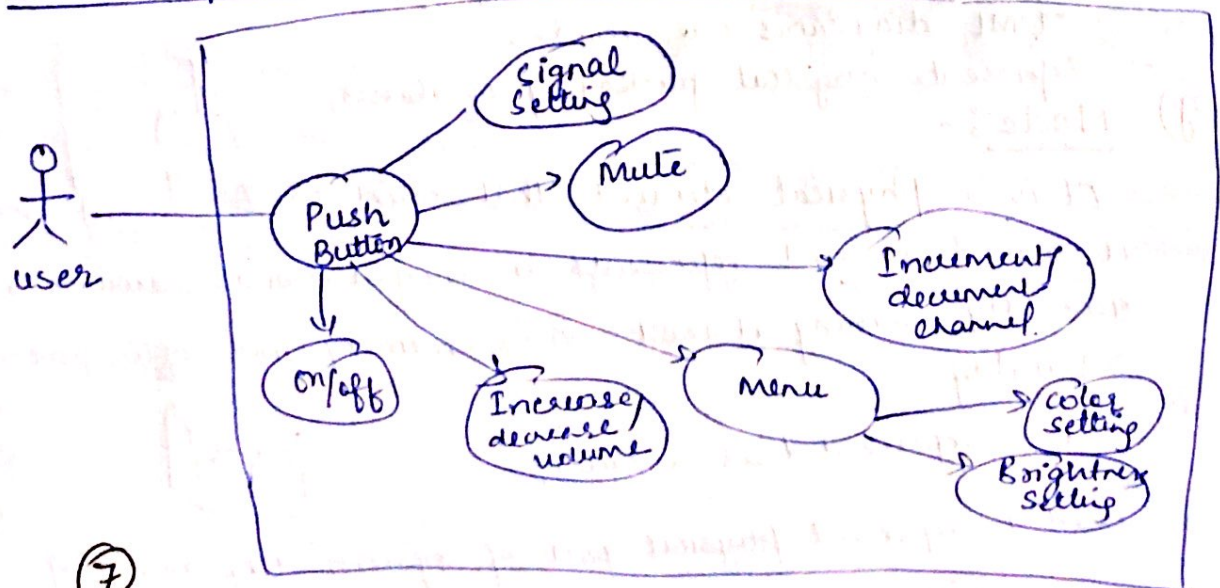
- A description of set of sequence of action that a system performs that yields to observable result of value to a particular actor.
- Use case represent the different way in which a system can be used by the users.
- A simple way to find all the use cases of a system is to ask the question..

" what the user can do using the system?"

ex, Super Market Prize scheme



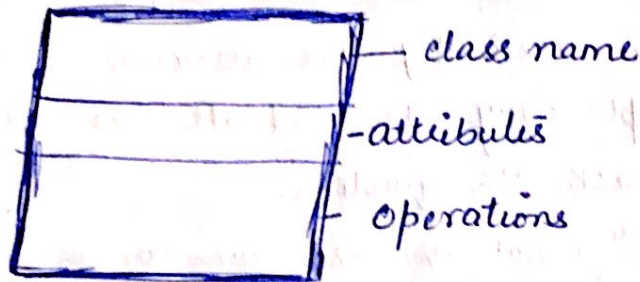
use case for TV remote system



(7)

### e) Active class

- Just like a class except that its object represent element whose behavior is concurrent with other elements.
- It is rendered like a class but with solid border.



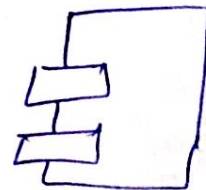
- It indicates that when instantiated, the class carries its own execution, rather than invoked or activated by other objects.
- Active class is a class where instances are active objects.
- It is generally used to describe the concurrent behavior of a system. It represent concurrency.

### f) Component describes physical part of a system

- It is a physical and replaceable part of a system that conforms to and provides the realization of a set of interfaces.
- It can be rendered as a rectangle with tabs
- It is used to represent any part of a system for which UML diagrams are made.
- Represents physical packaging of classes,

### g) Node:-

- It is a physical element that exist at run time and represents a computational resource, generally having at least some memory and often processing capability



→ It is represented as a cube



- used to represent physical part of system like server, n/w etc.

(8)

## h) Object :-

- Represented as rounded rectangle
- Active objects
  - ↳ exist in their own thread of operations & have their own address space.
  - ↳ It can change its state by itself.
- Passive objects
  - ↳ do not have ability to modify or begin the flow of execution, they must wait for another object to call.

## ② Behavioural things :-

- They are dynamic parts of UML models.
- These are the verbs of a model, representing behavior over time & space.
- Two primary kinds of behavioural things.
  - ① Interaction display messages
  - ② state machines waiting

a) Interaction Defined as a behaviour that consists of a group of messages exchanged among elements to accomplish a specific task - It is a behavior that comprises a set of messages exchanged among a set of objects within a particular context to accomplish a specific purpose.

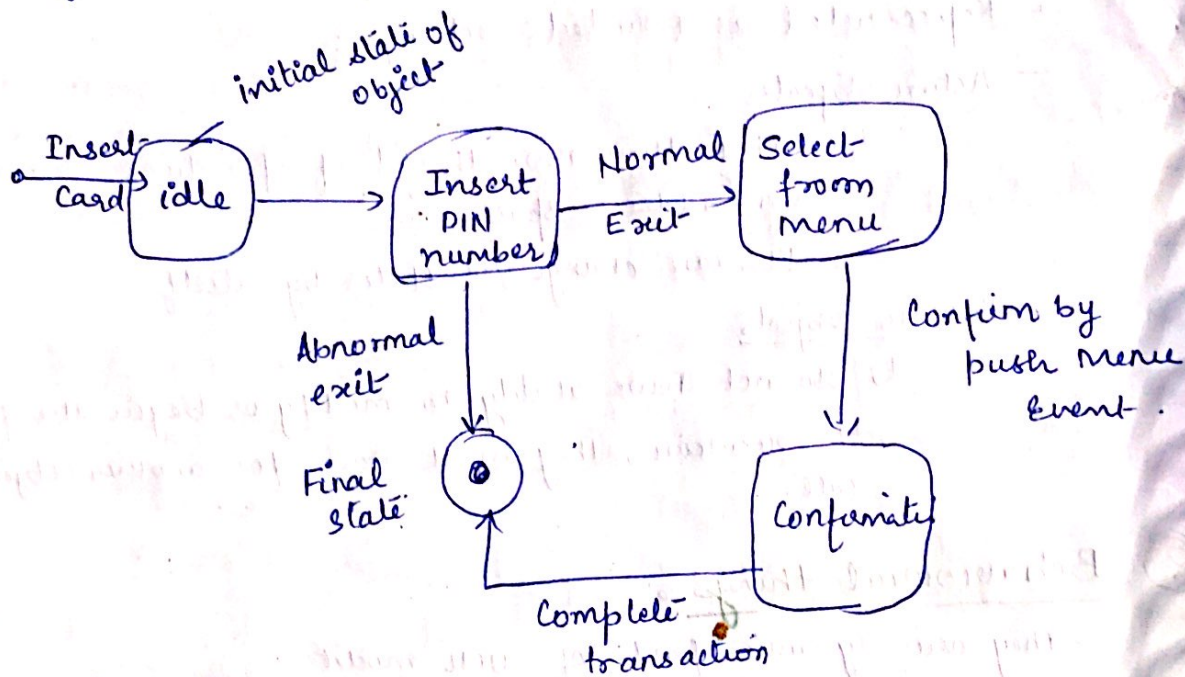
### b) State Machine

- It is a behavior that specifies the sequences of states of an object or an interaction goes through its lifetime in response to events, together with its responses to these events.

State

⑨

## Money withdrawal from ATM

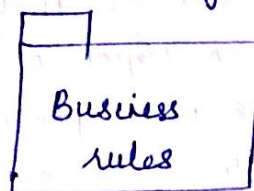


State machine used to describe different states of a system component. State can be active or idle.

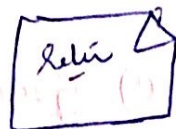
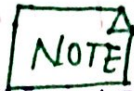
- ③ Grouping Things can be defined as a mechanism to group elements of UML model together.
- Are the organizational parts of UML models. These are the boxes into which a model can be composed.
  - One primary kind of grouping things is package.

### Package

A package is a general purpose mechanism for organizing elements into groups. Structural things, behavioral things, and even other grouping things may be placed in a package.



- ④ Annotational Things can be defined as mechanism to capture remarks, descriptions & comments of UML model elements. Annotational things are self explanatory parts of UML models.
- These are the comments you may apply to describe, illuminate, & remark about any element in a model. There is one primary kind of annotational thing, called a note.
  - A note is simply a symbol for rendering constraints and comments attached to an element or collection of elements.



→ RELATIONSHIPS - It shows how the elements are associated with each other and this association describes the functionality of an application.

There are 4 kinds of relationships in UML: -

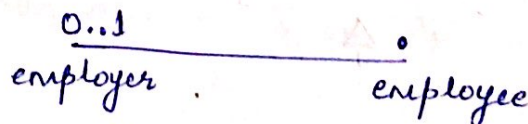
a) Dependency

- It is a semantic relationship b/w two things in which a change to one thing (the independent thing) may affect the semantics of the other thing (the dependent thing).

----->

b) Association

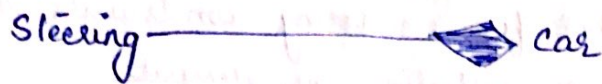
- It is a structural relationship that describes a set of links, a link being a connection among objects.
- Aggregation is a special kind of association, representing a structural relationship between a whole & its parts.



- Association defines the multiplicity between objects like one-to-one, many-to-one, one-to-many, many-many.
- It is represented as solid line having unfilled diamond at larger entity.

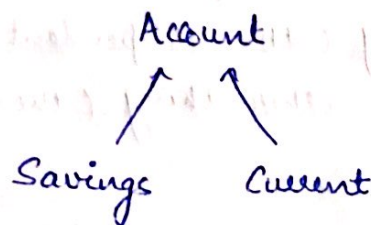


- composition : Aggregation & composition are whole part relationship. Composition is restricted aggregation. That is when a object contains another object and if contained object cannot exist without the existence of the container object, it is called composition



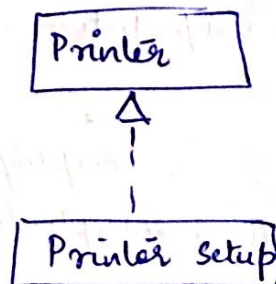
### c) Generalization

- Shows the parent child relationship
- child shares the behavior of the parent.
- Graphically it is represented by a solid line with a arrow head at one end.



### d) Realization

- It is relationship between model elements in which one model element realizes the behavior that the other model element specifies.
- It is represented by dashed line possibly directed & having label.



## ↳ DIAGRAMS

There are 9 diagrams in UML

### a) class Diagram

- shows a set of classes, interfaces & collaborations and their relationships.

- These diagrams are most common diagram found in modeling object oriented systems.
- It address the static <sup>design</sup> view of a system.
- Class diagrams that include active classes address the static process view of a system.

#### b) Object diagram

- Shows a set of objects and their relationships.
- Represents static snapshot of instances of the things found in class diagrams.
- It address the static design view or static process view of a system as do class diagrams, but from the perspective of real or prototypical cases.

#### c) Use Case diagram

- Shows a set of use cases and actors and their relationships.
- Address the static use view of a system.
- These diagrams are especially important in organizing & modeling the behaviors of a system.

#### d) Sequence diagram

- An interaction diagram that emphasizes the time ordering of messages,
- A collaboration diagram is an interaction diagram that emphasizes the structural organization of the objects that send & receive messages.
- Sequence diagram & collaboration diagram are isomorphic.

#### e) State chart diagram

- Shows state machine, consisting of states, transitions, events and activities.
- Address the dynamic view of system.
- Important in modeling the behaviors of an interface, class

## f) Activity Diagram

- Special kind of state chart diagram that shows the flow from activity to activity within a system.
- Address the dynamic view of a system.
- Important in modeling the function of a system.
- Emphasize the flow of control among objects.

## g) Component Diagram

- shows the organizations and dependencies among a set of components.
- Address the static implementation view of system.
- Related to <sup>class</sup> component diagrams and in that it typically maps to one or more classes.

## h) Deployment Diagram

- shows the configuration of run-time processing nodes and the components that live on them.
- Address the static deployment view of an architecture.
- Related to component diagrams in that a node typically encloses one or more components.

## UML RULES:-

Rules are defined for names, scope, visibility, integrity and execution.

### 1) Name

- Represents things, relationships and diagrams.

### 2) Scope

- Represents some text that gives a specific meaning to names.

### 3) Visibility

- shows the names that can be seen and used by others.

#### 4) Integrity

- Shows how properly and consistently the entities can be related to one another.

#### 5) Execution

- Represents the running or implementation of the model.

### COMMON MECHANISMS IN UML: -

There are 4 common mechanisms in UML.

#### 1) Specification

- Along with graphics UML provides textual statements that have some special semantic and syntax.
- For example, along with the class, one can specify the attributes and operations carried out by that class.

#### 2) Adornments

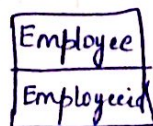
- Graphical notations of UML that specify the things in more detail. It allows the user to specify extra information with basic notation of an element.
- Helps in proper visualization of a system.
- For ex, along with rectangular graphical representation, we can specify the attributes and operations of a class in detail.

#### 3) Common Divisions

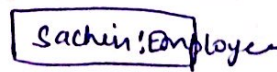
In UML there is a common division b/w some objects.

a) Classes and objects class is an abstraction of object and object is an instance of a class.

For. a class can be Employee and objects can be Sachin.

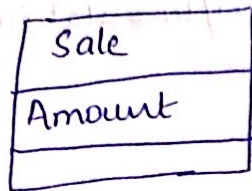


class



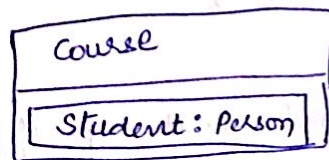
Object.

b) Interfaces and Implementations : The interface represents a contract between two entities, whereas the implementation represents the concrete realization of the contract. The interface's complete semantic can be defined by the 'implementation'.

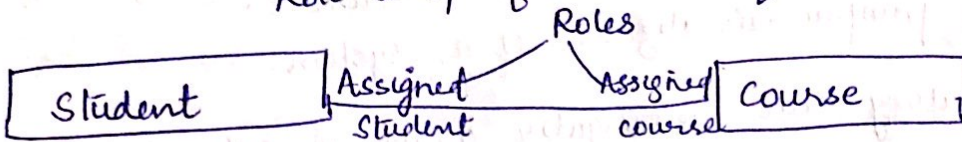


c) Roles and Types : The type represents the class of an entity such as object, attribute, and parameters and so on. The role defines the meaning of an entity with respect to class.

- One entity can be derived from other & takes characteristics from its parents.



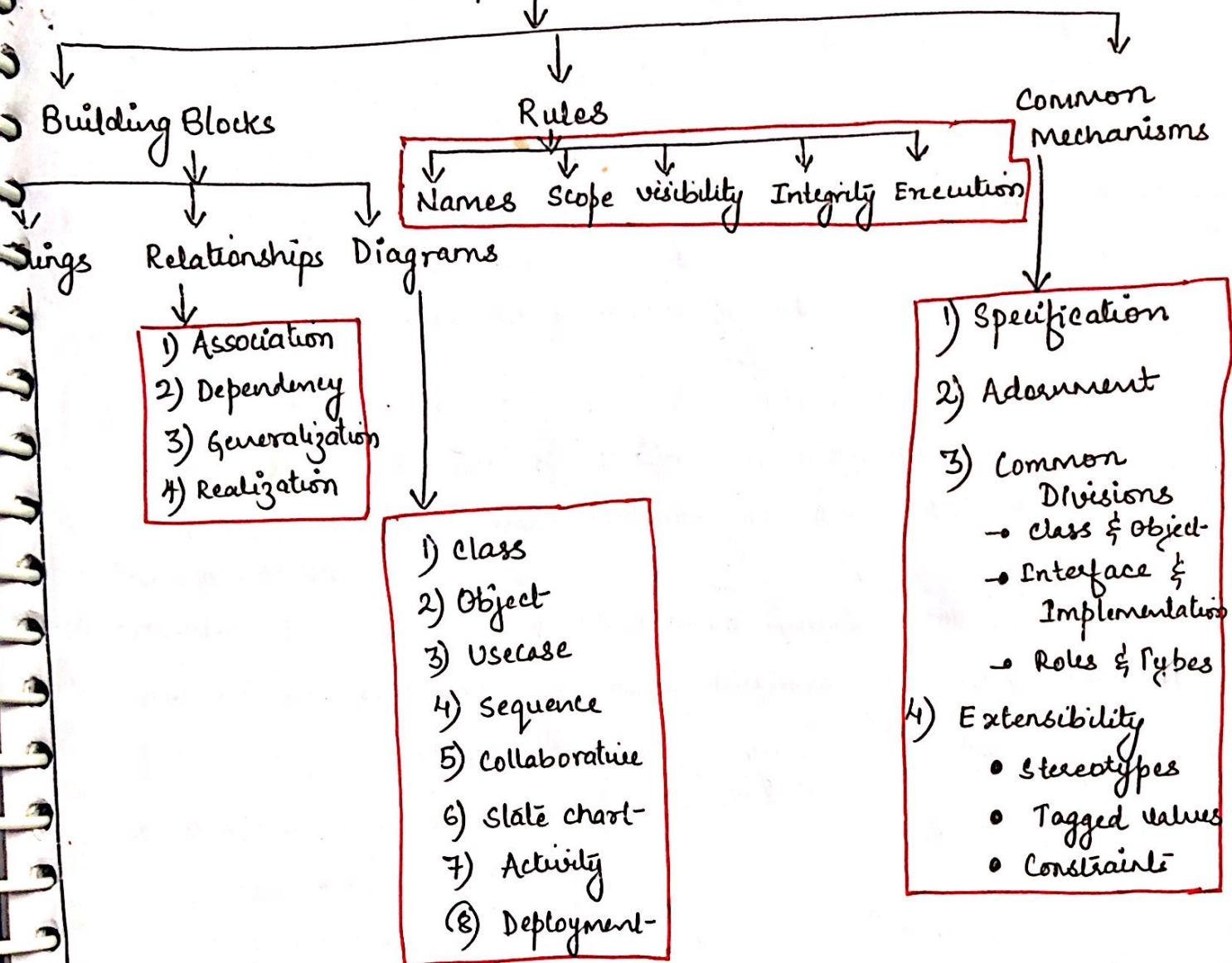
Role is specifies at end of relation



#### 4) Extensibility

- Sometimes it is not possible to represent all the refinements of the system using the model.
- For this model reason, UML is open-ended.
- we can extend the language in controlled ways.
- It includes stereotypes, tagged value and constraints.
- The extensible mechanism in UML are
  - i) Stereotypes
  - ii) Tagged value
  - iii) Constraints

# Conceptual Model of UML



## Structural Things

- 1) Class
- 2) Object
- 3) Collaboration
- 4) Use case
- 5) Interface
- 6) Component
- 7) Node
- 8) Active class

## Behavioural Things

- 1) Interaction
- 2) State Machine

## Grouping Things

Package

## Annotational Things

Notes